Hossain Mohammad Abujar
IT-22056

1. Write a program to find the kth smallest element in a arraylist.

```
import java.util.*;
Public class kthSmallest {
    Public static int findkthSmallest(List<Integer>list,
                                      int k){
        collections.sort(list);
        return list.get(k-1);}
public static void main(String[] args){
    List<Integer> numbers = Array.asList(7,10,4,3,20)
    int k=3;
    int result = findkthSmallest(numbers, k);
    System.out.println("the "+k+" nd smallest element
                       is :" + result); }}
```

2. Create a tree map to store the mapping of words to their frequencies in a given text.

```
import java.util.*;
Public class wordFrequencyMap{
  public static void main(String args[]){
    String text = "hello world hello Java";
```

```
String[] words = text.split(" ");
TreeMap<String, Integer> frequencyMap = new TreeMa
    for (String word : words)
{// update frequency count
frequencyMap.put(word, frequencyMap.getOr
Default (word,0)+1);}
// Display word frequencies in sorted order.
 System.out.println("word Frequencies:");
for (Map Entry<String, Integer> entry : frequenc
          MP.entry set()){
 System.out.println(entry.get key()+" "+entry.
                    getValue());
  }
 }
}
```

3. Queue and stack using priority queue with a custom comparator.

```
import java.util.*;
Public class QueueStackPg {
Public static void main(String[] args) {
//Queue as Min-heap
PriorityQueue<Integer> queue = new PriorityQueue<>();
        queue.add(3);
        queue.add(1);
        queue.add(2);
System.out.println("Queue (ascending order): ");
 while (!queue.isEmpty()){
 System.out.print(queue.poll()+"  ");
 }
// Stack using Max-heap
PriorityQueue<Integer> stack = new PriorityQueue<>
                    (Collection.reverseOrder());
      stack.add(1);
      stack.add(2);
      stack.add(3);
System.out.println("\n Stack (descending order): ");
 while (!stack.empty()){
  System.out.print(stack.poll()+"  "); }}}
```

4. Treemap to store student IDs and their details.

```
import. Java. util. *;
// create a Student class to hold student details.
  class Student{
        String name;
         int age;
  Student ( String name, int age){
        this. name = name;
        this. age = age; }
 Public String toString (){
  return name + "(" +age+"); }
  public string tostring (){
    return name +" ("+age+");
  }
 }
public class StudentMap{
 public static void main (String [] args){
 TreeMap <Integer, student> Students = new TreeMap(
 Students. put (101, new student ("Adam", 20);
 Student. put (102, newstudent ("Arin", 21);
 // print each Student ID and details
```

```
for (Map.Entry<Integer, Student> entry : students
                .entrySet()){
System.out.println("ID "+entry.getKey()+",
    Details : "+entry.getValue());
}
}
}
```

5. Check if two linkedlists are equal.

```
import java.util.*;
public class LinkedListEqual {
public static void main (String[] args){
// create two linked lists with the same values
LinkedList<Integer> list1 = new LinkedList<>
            (Arrays.asList(1,2,3));
LinkedLisk<Integer> list2 = new LinkedList<>
            (Arrays.asList(1,2,3));
//use equal method to compare
boolean isEqual = list1.equals(list2);
System.out.println("Are the lists equal 2" +isEqual);
}
}
```

6. EmployeeDepartment Map Java.

```
import java.util.*;
Public class EmployeeDepartmentMap {
Public static void main (String[] args){
HashMap<Integer, String> employeeMap = new
                    HashMap <>();
employeeMap.put (10001, "HR");
employeeMap. put ( 10002, "IT");
employeeMap . put (10003, "Finance");
for (Map.Entry <Integer, String> entry : employee
 Map entrySet ()) {
 system.out.println ("ID" + entry. getKey () + "=>
   Dept:" + entry.getValue());
 }
}
```